Mineração de Métricas DORA em Repositórios Open-Source: Quanto Podemos Confiar nas Aproximações?

Arthur Lara Panzera[1], **Felipe Augusto Pereira**[1], **Gabriel Reis Lebron de Oliveira**[1]

[1]Curso de Engenharia de Software – PUC Minas
Belo Horizonte – MG – Brasil

arthurpanzera@outlook.com, felipeaps0918@gmail.com, reislebron@gmail.com

Abstract. *DORA metrics (deployment frequency, lead time for changes, change failure rate and failed deployment recovery time) are the industry standard for measuring software delivery performance. GitHub, however, records neither production deployments nor production failures, so these metrics can only be approximated from releases, commits and CI workflow runs. This paper mines DORA metrics from popular open-source repositories that use GitHub Actions, validates the automatic criteria against human judgment, and measures how much each repository's DORA classification depends on the chosen operational definition.*

Resumo. *As métricas DORA (frequência de deploy, lead time de mudanças, taxa de falha de mudanças e tempo de recuperação) são o padrão de mercado para medir o desempenho de entrega de software. O GitHub, porém, não registra deploys nem falhas em produção, de modo que essas métricas só podem ser aproximadas a partir de releases, commits e execuções de CI. Este trabalho minera as métricas DORA de repositórios open-source populares que usam GitHub Actions, valida os critérios automáticos contra o julgamento humano e mede o quanto a classificação DORA de cada repositório depende da definição operacional escolhida.*

1. Introdução

Medir o desempenho de entrega de software é uma preocupação antiga da Engenharia de Software. Nos últimos anos, as métricas propostas pelo programa DORA (*DevOps Research and Assessment*), popularizadas pelo livro *Accelerate* [Forsgren et al. 2018] e revisadas anualmente pelos relatórios *Accelerate State of DevOps* [DORA 2024], tornaram-se o padrão de mercado para esse fim. Elas avaliam duas dimensões: a *velocidade*, por meio da frequência de deploy (*deployment frequency*) e do tempo entre um commit e sua entrega em produção (*lead time for changes*), e a *estabilidade*, por meio da proporção de deploys que causam falha (*change failure rate*, CFR) e do tempo para se recuperar delas (*failed deployment recovery time*) [DORA a]. Em 2024, o DORA acrescentou uma quinta métrica, a *deployment rework rate* [DORA b].

Essas métricas foram concebidas para equipes que operam serviços em produção e são, em geral, coletadas por questionários ou por ferramentas internas das organizações. Repositórios open-source oferecem uma oportunidade de medi-las em larga escala e com dados públicos, mas impõem um obstáculo: o GitHub não registra "deploys em produção" nem "falhas em produção". O que existe são *releases*, *commits* e execuções de *workflows*

de integração contínua. Qualquer cálculo das métricas DORA nesse contexto depende, portanto, de *proxies*: uma release publicada é tomada como um deploy, e uma execução de CI que falhou é tomada como uma falha. Esses proxies podem enganar. Uma biblioteca que publica uma release por mês não está colocando nada em produção; quem o faz são seus usuários.

Este trabalho minera as métricas DORA de repositórios open-source populares (com mais de 1.000 estrelas) que usam GitHub Actions, em uma janela de observação de 12 meses (de 1º de outubro de 2025 a 30 de setembro de 2026). Além de calcular as métricas, o estudo questiona a confiabilidade dos próprios valores obtidos: os critérios automáticos são comparados com o julgamento de três avaliadores humanos sobre uma amostra de 60 repositórios, e uma análise de sensibilidade mede o quanto a classificação DORA de cada repositório muda quando a definição operacional da métrica muda. A pergunta deixa de ser apenas "qual é o valor da métrica?" e passa a ser também "o quanto podemos confiar nesse valor?".

O estudo é guiado pelas sete questões de pesquisa (RQs) a seguir. Para cada uma, registramos uma hipótese informal, isto é, o que esperamos encontrar e por quê. As hipóteses foram escritas **antes** da coleta e da análise dos dados, para que a comparação com os resultados, feita na Seção 4, não seja influenciada por eles.

RQ1. Qual a frequência de deploys dos repositórios populares que usam CI/CD?
Hipótese H1: esperamos uma mediana entre uma release por mês e uma release por semana, o que corresponde à faixa *Medium* da tabela de referência, com distribuição fortemente assimétrica à direita. Projetos open-source costumam agrupar mudanças em versões planejadas, e muitos deles são bibliotecas, cujo ciclo de publicação é mais lento que o de serviços. Esperamos poucos repositórios *Elite* (sete ou mais releases por semana), concentrados em projetos que automatizam a publicação a cada merge.

RQ2. Qual o tempo entre um commit e seu respectivo deploy?
Hipótese H2: esperamos lead times da ordem de semanas, e não de horas, classificando a maioria dos repositórios como *Medium* ou *Low*. Como a release é publicada em lotes, um commit espera até a próxima versão para ser entregue. Esperamos ainda que a variante por release (a), baseada no commit mais antigo, seja sistematicamente maior que a variante por commit (b), baseada na mediana de todos os commits: um único commit antigo "esquecido" em uma release eleva muito a variante (a) e pouco afeta a (b).

RQ3. Qual a taxa de falha das mudanças entregues por esses repositórios?
Hipótese H3: para a variante (a), proxy de CI, esperamos uma mediana entre 10% e 25%. Falhas de pipeline são frequentes em projetos ativos por causas que não chegam à produção, como testes instáveis (*flaky*), erros de lint e falhas de infraestrutura. Para a variante (b), proxy de entrega, esperamos valores menores, abaixo de 15%, pois uma release corretiva publicada em até sete dias exige que um defeito tenha sido percebido e corrigido rapidamente. Esperamos também baixa concordância entre as duas variantes, já que elas medem fenômenos diferentes: falha de pipeline e defeito entregue ao usuário.

RQ4. Qual o tempo de recuperação após uma execução de CI/CD com falha?
Hipótese H4: esperamos uma mediana entre uma hora e um dia (faixa *High*). Boa parte das falhas de CI é corrigida rapidamente, com um novo commit ou após uma

falha transitória, mas esperamos uma cauda longa de episódios que duram dias, em workflows menos prioritários, como os de documentação ou de publicação de pacotes. Esperamos uma proporção pequena de episódios censurados, abaixo de 10% por repositório.

RQ5. Repositórios com maior frequência de deploy apresentam maior ou menor taxa de falha?
Hipótese H5: em linha com a afirmação do DORA de que velocidade e estabilidade não são um *trade-off*, esperamos uma correlação de Spearman fraca ($|\rho| < 0{,}3$) entre a frequência de deploy e o CFR, nas duas variantes. Para o CFR (b), esperamos uma correlação levemente positiva, pois quem publica com mais frequência tem mais oportunidades de seguir uma release com uma correção em até sete dias, o que pode ser um artefato da própria definição, e não um sinal de menor estabilidade.

RQ6. Quais características dos repositórios estão associadas a um melhor desempenho DORA?
Hipótese H6: esperamos que o tipo do projeto e o número de contribuidores sejam os fatores com maior associação. Aplicações, serviços e ferramentas CLI devem publicar com mais frequência e ter lead times menores que bibliotecas e frameworks, que priorizam a estabilidade da API. Repositórios com mais contribuidores devem se recuperar mais rápido de falhas de CI, por terem mais pessoas disponíveis para corrigi-las. Para popularidade e linguagem, esperamos diferenças pequenas ou desprezíveis após a correção de Holm para comparações múltiplas.

RQ7. O quanto a classificação DORA de um repositório depende da definição operacional escolhida?
Hipótese H7: esperamos que a classificação seja sensível à definição: que pelo menos 30% dos repositórios mudem de categoria entre a combinação de referência e as alternativas, com kappa de Cohen ponderado apenas moderado (entre 0,41 e 0,60). Esperamos a maior mudança ao incluir pré-releases ou tags como unidade de deploy, que pode elevar a frequência de deploy de vários repositórios em uma ou mais faixas.

As principais contribuições deste trabalho são: (i) um pipeline reprodutível de mineração das métricas DORA a partir da API do GitHub; (ii) um conjunto de dados com as métricas de mais de 300 repositórios, acompanhado de dicionário de dados; (iii) a validação dos critérios automáticos contra uma amostra rotulada por humanos; e (iv) uma análise de quão robustas são as conclusões frente à escolha da definição operacional.

O restante do artigo está organizado da seguinte forma: a Seção 2 descreve a coleta, as definições operacionais e o protocolo de validação manual; a Seção 3 apresenta os resultados por questão de pesquisa; a Seção 4 compara as hipóteses com os resultados; a Seção 5 discute as ameaças à validade; e a Seção 6 relata a replicação cruzada.

2. Metodologia

Esta seção descreve como os repositórios foram selecionados, como os dados foram coletados, como cada métrica foi calculada a partir deles e como os critérios automáticos foram validados contra o julgamento humano. Todo o processo é executado por um único pipeline, disponível no repositório do grupo, que lê a configuração de um arquivo (janela, faixas de estrelas, critérios de inclusão e sementes dos sorteios) e grava os dados brutos,

as métricas e os resultados da validação em arquivos CSV, descritos em um dicionário de dados.

2.1. Fonte de dados e coleta

Os dados foram obtidos das APIs REST e GraphQL do GitHub, por um cliente HTTP próprio, sem bibliotecas prontas de acesso à API. Cada resposta é guardada em um cache local (SQLite), o que permite retomar uma coleta interrompida sem repetir chamadas. O cliente respeita o *rate limit* pelos cabeçalhos `X-RateLimit-Remaining` e `X-RateLimit-Reset` e repete as requisições que falham por erros temporários com espera exponencial.

Para cada repositório da amostra foram coletados os metadados (estrelas, linguagem, data de criação, número de contribuidores e *default branch*), as releases, as tags, os commits entre releases consecutivas e os *workflow runs*. Releases vêm da API REST. Tags e comparações entre releases vêm da API GraphQL, que traz a data do commit apontado por cada tag e a mensagem completa dos commits em uma única consulta. A API GraphQL, porém, limita a comparação a 1.000 commits; quando o par de releases tem mais que isso, a comparação é refeita pelo *endpoint* `compare` da API REST, com paginação, para que nenhum commit fique de fora.

Os *workflow runs* foram coletados apenas no *default branch* e disparados por `push`. Como a listagem de runs devolve no máximo 1.000 resultados por consulta, a janela foi dividida em meses, e cada mês que atingia o teto foi dividido ao meio até caber nele. Ao todo foram feitas 3.872 consultas, e nenhuma ficou truncada, o que resultou em 528.304 runs nos 300 repositórios.

2.2. Seleção dos repositórios

A janela de observação, definida pelo professor, vai de 1º de outubro de 2025 a 30 de setembro de 2026, em UTC. Só entram no cálculo as releases publicadas e os runs criados dentro dela.

Os candidatos foram obtidos pela busca de repositórios do GitHub, restrita a projetos com pelo menos 1.000 estrelas que não são *forks* nem estão arquivados. A busca devolve no máximo 1.000 resultados por consulta, os de mais estrelas, e por isso foi fatiada em cinco faixas (1.000 a 2.000, 2.000 a 5.000, 5.000 a 10.000, 10.000 a 50.000 e acima de 50.000 estrelas). Mesmo assim, as faixas mais baixas têm dezenas de milhares de repositórios, e uma consulta por faixa traria apenas os de mais estrelas de cada uma. Cada faixa foi então dividida ao meio, recursivamente, até que cada pedaço coubesse no teto da busca. Assim obtivemos a lista completa de 60.043 candidatos, que foi congelada em arquivo para que todas as execuções seguintes, e o grupo replicador, partam da mesma população.

Os candidatos foram avaliados em uma ordem aleatória e reproduzível, dada pelo *hash* SHA-256 do nome de cada repositório combinado a uma semente fixa, e a amostra foi formada pelos primeiros 300 que passaram nos critérios. Isso equivale a sortear 300 repositórios entre os elegíveis, sem precisar avaliar todos os 60 mil. Os critérios foram aplicados do mais barato para o mais caro, parando no primeiro que o repositório não atendia: usar GitHub Actions (`total_count` maior que zero em `/actions/workflows`), ter

pelo menos cinco releases na janela e ter pelo menos 50 runs válidos na janela, isto é, com sucesso ou falha. A Tabela 1 mostra o funil resultante.

Tabela 1. Funil de seleção dos repositórios.

Etapa	Restantes	Descartados
Candidatos (1.000 estrelas ou mais)	60.043	–
Avaliados até completar a amostra	2.160	57.883
Usam GitHub Actions	1.238	922
Pelo menos 5 releases na janela	411	827
Pelo menos 50 runs válidos na janela	300	111

A amostra final tem 96 repositórios na faixa de 1.000 a 2.000 estrelas, 98 na de 2.000 a 5.000, 49 na de 5.000 a 10.000, 56 na de 10.000 a 50.000 e um acima de 50.000, com mediana de 3.310 estrelas. As linguagens mais frequentes são TypeScript (76 repositórios), Go (40), Rust (38) e Python (35).

2.3. Definições operacionais

Seguimos as definições comuns da disciplina, para que os resultados sejam comparáveis entre os grupos. A unidade de entrega (*deploy*) é uma release publicada, com `draft` igual a `false`; pré-releases e tags sem release ficam fora da definição principal e são usadas apenas como variantes na RQ7. A data de um commit é a data de autor (`commit.author.date`), e a release anterior a uma release é a publicada imediatamente antes dela. Um run é considerado sucesso quando sua `conclusion` é `success` e falha quando é `failure`, `timed_out` ou `startup_failure`; os demais valores, como `cancelled` e `skipped`, são ignorados em todos os cálculos.

Frequência de deploy (RQ1). Número de releases na janela dividido pelo número de semanas da janela (cerca de 52,1).

Lead time (RQ2). Para cada release, consideramos os commits incluídos nela, isto é, os da comparação com a release anterior. Na variante (a), o lead time da release é a diferença entre sua publicação e o commit mais antigo incluído, e o valor do repositório é a mediana entre as releases. Na variante (b), calculamos a diferença entre a publicação e cada commit, e o valor do repositório é a mediana de todos os commits de todas as releases. Ficam fora do cálculo a primeira release da história do repositório, que não tem anterior para comparar, as releases cuja comparação falhou (por exemplo, quando não há ancestral comum entre as duas tags) e os commits com data de autor posterior à própria release, que aparecem após *rebases* e *cherry-picks*. Esses casos são contados e registrados no conjunto de dados.

Taxa de falha (RQ3). Na variante (a), proxy de CI, o CFR é o número de runs com falha dividido pela soma de falhas e sucessos. Na variante (b), proxy de entrega, uma release falha quando é seguida, em até sete dias, por uma release corretiva, e o CFR é a proporção de releases que falharam entre as avaliadas. As releases publicadas nos últimos sete dias da janela não podem ser avaliadas, pois ainda não se sabe se haverá correção, e ficam fora do denominador. A heurística que identifica uma release corretiva é descrita na Seção 2.4.

Também separamos as falhas intermitentes (*flaky*), em que o CI falha e depois passa no mesmo commit, sem mudança de código. Uma falha seguida de um sucesso no mesmo *workflow* e no mesmo commit (`head_sha`) é retirada de uma versão alternativa do CFR (a), mantida ao lado da original para comparação. Contamos ainda os runs reexecutados (`run_attempt` maior que um) que terminaram em sucesso. A API, no entanto, só lista a última tentativa de cada run, de modo que a falha que motivou a reexecução não aparece nos dados e o run já entra no CFR original como sucesso.

Tempo de recuperação (RQ4). Os runs de cada *workflow* são ordenados por `run_started_at`. Um episódio de falha começa na primeira falha após um sucesso e termina na próxima execução bem-sucedida do mesmo *workflow*, e sua duração vai do início da falha (`run_started_at`) ao fim do sucesso (`updated_at`). O valor do repositório é a mediana dos episódios. Episódios que não terminam até o fim da janela não são descartados: são registrados como censurados, e a proporção de censurados de cada repositório é informada. Falhas anteriores ao primeiro sucesso de um *workflow* não abrem episódio, pois o momento em que a falha começou é desconhecido.

Classificação DORA. Cada métrica é classificada como *Elite*, *High*, *Medium* ou *Low* pelos cortes fixos da disciplina, adaptados dos relatórios DORA. A classificação geral atribui de 4 a 1 ponto a cada categoria e toma a mediana das quatro notas, arredondada para baixo. Na combinação de referência (C1), usada nas RQ1 a RQ6, a unidade de deploy é a release, e o lead time e o CFR são os da variante (a). Quando o repositório não teve nenhuma falha de CI, não há tempo de recuperação, e a classificação geral é a mediana das três métricas restantes.

Sensibilidade (RQ7). Para medir o quanto a classificação depende da definição, comparamos C1 com duas combinações alternativas: C2, com releases e pré-releases como unidade de deploy e as variantes (b) de lead time e CFR, e C3, com tags como unidade de deploy, lead time (b) e CFR (a). A data de uma tag é a data de autor do commit para o qual ela aponta. Para cada par de combinações, calculamos a proporção de repositórios que mudaram de categoria e o kappa de Cohen ponderado linearmente, que pune menos uma troca entre categorias vizinhas do que entre categorias distantes.

2.4. Validação manual

Para saber o quanto os critérios automáticos erram, comparamos esses critérios com o julgamento humano em uma amostra-ouro. Sorteamos 60 dos 300 repositórios e, de cada um, cinco releases da definição principal publicadas na janela, totalizando 300 releases, com semente fixa e documentada no código. Cada um dos três integrantes do grupo rotulou a amostra sozinho, sem consultar os demais e sem ver a saída das heurísticas, em uma planilha gerada automaticamente com os links para cada repositório, release e lista de commits. Para cada repositório, o avaliador indicou o tipo do projeto (biblioteca/framework, aplicação/serviço, ferramenta CLI ou outro) e se as releases representam entregas reais ao usuário (sim, não ou incerto); para cada release, indicou se ela é corretiva, ou seja, se seu objetivo principal é corrigir um defeito recente.

A concordância entre os três foi medida com o kappa de Fleiss, em cada uma das três dimensões, e interpretada pela escala de Landis e Koch [Landis and Koch 1977]. O rótulo de consenso seguiu um protocolo definido antes da discussão: o rótulo unânime é o consenso; sem unanimidade, vale a maioria simples (dois contra um); e os casos em que

os três divergem são decididos em reunião e registrados no repositório. A Tabela 2 mostra a concordância obtida. O kappa de *entregas reais* ficou baixo apesar da concordância observada de 94%, porque quase todos os repositórios foram rotulados como *sim*: com uma categoria tão dominante, a concordância esperada por acaso já é alta, e cada divergência pesa muito no kappa. Houve um único caso sem maioria, nessa mesma dimensão.

Tabela 2. Concordância entre os três avaliadores na amostra-ouro.

Dimensão	Itens	Concordância	Kappa	Interpretação
Tipo do projeto	60	92,2%	0,88	quase perfeita
Entregas reais	60	93,9%	0,29	razoável
Release corretiva	300	84,9%	0,69	substancial

O consenso serviu de referência para avaliar a heurística de release corretiva, com precisão, recall e F1, sendo a release corretiva a classe positiva. Partimos das sugestões do enunciado e refinamos a heurística a cada rodada, olhando os erros da versão anterior, até superar o F1 mínimo de 0,70 (Tabela 3). As versões que dependem da release anterior (v1 e v2) erraram em projetos com várias linhas de versão, em que a release publicada logo antes pode ser de outra linha. Por isso as versões seguintes olham apenas a própria versão. Adotamos a v3, de maior F1, que considera corretiva toda release cuja versão é um *patch* (X.Y.Z com Z maior que zero). Seu recall é alto, mas a precisão de 0,58 indica que ela também marca como corretivas versões de *patch* que trazem outras mudanças, o que tende a inflar o CFR (b).

Tabela 3. Versões da heurística de release corretiva frente ao consenso (300 releases, 126 corretivas). Commit de correção é aquele cuja mensagem contém *fix*, *hotfix* ou *revert*.

Versão	Regra	Precisão	Recall	F1
v1	Só o *patch* muda em relação à anterior OU há commit de correção	0,47	0,96	0,63
v2	Só o *patch* muda em relação à anterior E há commit de correção	0,68	0,61	0,64
v3	A própria versão é um *patch* (X.Y.Z com $Z > 0$)	0,58	0,94	0,72
v4	A própria versão é um *patch* E há commit de correção	0,67	0,76	0,71

O tipo do projeto, usado na RQ6, foi rotulado por humanos apenas na amostra-ouro. Para os demais 240 repositórios, ele foi atribuído por uma classificação automática baseada em palavras-chave da descrição, dos tópicos e do início do README de cada repositório. Validada contra o consenso, a primeira versão acertou 73% dos 60 repositórios; após ajustar os padrões com base nos erros, a segunda versão chegou a 95% de acurácia, com F1 macro de 0,93. Nos repositórios da amostra-ouro, prevalece o rótulo humano.

3. Resultados

4. Discussão

5. Ameaças à Validade

6. Replicação Cruzada

Disponibilidade

Repositório do grupo: `https://github.com/Lab-Experimentao/Lab03-MineracaoMetricasDora`.
GitHub Projects do grupo: `https://github.com/orgs/Lab-Experimentao/projects/2/views/2`.

Referências

DORA. DORA's software delivery metrics: the four keys. `https://dora.dev/guides/dora-metrics-four-keys/`. Acesso em: out. 2026.

DORA. A history of DORA's software delivery metrics. `https://dora.dev/insights/dora-metrics-history/`. Acesso em: out. 2026.

DORA (2024). Accelerate state of DevOps report. `https://dora.dev/research/`. Acesso em: out. 2026.

Forsgren, N., Humble, J., and Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution.

Landis, J. R. and Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174.